

- Ahmed
- HASEEB ~~Amer~~ Chattha.
 - 70136897.
 - BSCS-4E
 - DSA

Breadth-First search (BFS):

BFS is an algorithm used to traverse a tree or a graph in a breadth-first manner, which means it explores all the vertices at the same level before moving on to the next level. It starts from a specific vertex. BFS uses Queue data structure to keep track of vertices that have been visited but not processed.

Code:

```
Class Node {  
    public:  
        int data;  
        Node *left, *right;  
        Node (int value) {  
            data = value;  
            left = NULL;  
            right = NULL;  
        }  
};
```



```
void printCurrentLevel(Node* root, int level);
int height(Node* node);
```

```
void printLevelOrder(Node* root) {
    int h = height(root);
    for (int i = 1; i <= h; i++) {
        printCurrentLevel(root, i);
    }
}
```

```
void printCurrentLevel(Node* root, int level) {
    if (root == NULL) {
        return;
    }
    if (level == 1) {
        root->data << " ";
    }
    else if (level > 1) {
        printCurrentLevel(root->left, level-1);
        printCurrentLevel(root->right, level-1);
    }
}
```

```
int height(Node* node) {
    if (node == NULL) {
        return 0;
    }
    else {
        int lheight = height(node->left);
        int rheight = height(node->right);
        return (lheight > rheight) ? (lheight+1) : (rheight+1);
    }
}
```

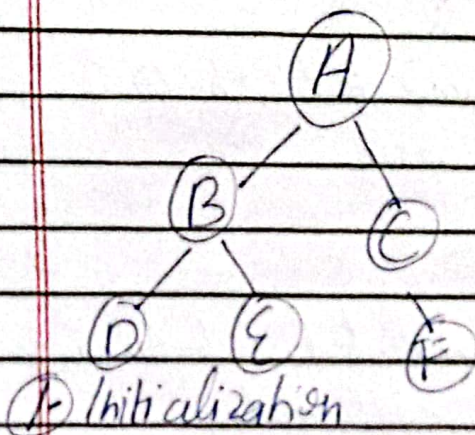


```

int main() {
    Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);
    root->left->left = new Node(4);
    root->left->right = new Node(5);
    printLevelOrder(root);
    return 0;
}

```

DRY RUN :-



1) Initialization

- Create an empty queue
- Enqueue the root node ('A') and mark it as visited

2) Iteration:

- Dequeue the front node from the queue ('A')

- Enqueue its unvisited children (B & C) and mark them as visited

3) Iteration:

- Dequeue the front node from the queue (B)
- Enqueue its unvisited children (D & E) and

DATE: 1/120

MTWTFSS

mark mem as visited

4- Iteration

→ dequeue C

→ enqueue its child (F) & mark it visited

5- Iteration:

→ dequeue (D)

→ Since (D) has no unvisited children we move to next node

6- Iteration:

→ dequeue (E)

→ Since E has no unvisited children we move to next node

7- Iteration:

→ dequeue F.

→ Since F has no unvisited children, we move to the next node

8- Termination:

→ The queue is empty indicating that we have visited all nodes.

DATE: 1/20

Queue demonstration:

→ [] empty

→ [A] → enqueue

→ {A} → visited

Iteration 1:

→ dequeue A, enqueue B, C

→ [B, C]

→ visited {A, B, C}

Iteration 2:

→ dequeue B, enqueue D, E

→ [C, D, E]

→ visited {A, B, C, D, E}

Iteration 3:

dequeue C, enqueue F

[D, E, F] {A, B, C, D, E} → visited

Iteration 4:

dequeue D [E, F] //

Iteration 5:

dequeue E: [F] //

Iteration 6:

dequeue F: [] //

Iteration 7 / termination

All nodes have been visited